Universidad Internacional Del Ecuador

Name: Luis Francisco Quilumbaquin Chavez

Date: 14/05/2026

1. Theoretical Framework

Technical comparison: Why shouldn't fast algorithms like MD5 or SHA-256 be used for passwords?

Look, using algorithms like MD5 or SHA-256 for passwords is a bad idea because they're too fast. They're primarily designed to check that a downloaded file isn't corrupted. If we use them to store passwords, any hacker with a reasonably powerful computer can try to guess millions of combinations in a second (what's known as brute force). That's why it's better to use things like Bcrypt, which are intentionally "slow" algorithms. This way, if someone tries to force a password, the computer will take an eternity for each attempt, making it virtually impossible.

Explanation of the data flow: At what point is Pepper added and why is it not saved in the database?

Pepper is basically a secret word that we append to the password the user enters right before encrypting (hashing) it. Importantly, we never store this in the database; we keep it hidden on the server (in a file called .env). If we stored everything together and the database were hacked, the thief would get both the lock and the key. Since we keep it separate, if they steal the .db file, it's useless to them because they'll lack that secret part.

2. Development and Evidence

SQLModel configuration.

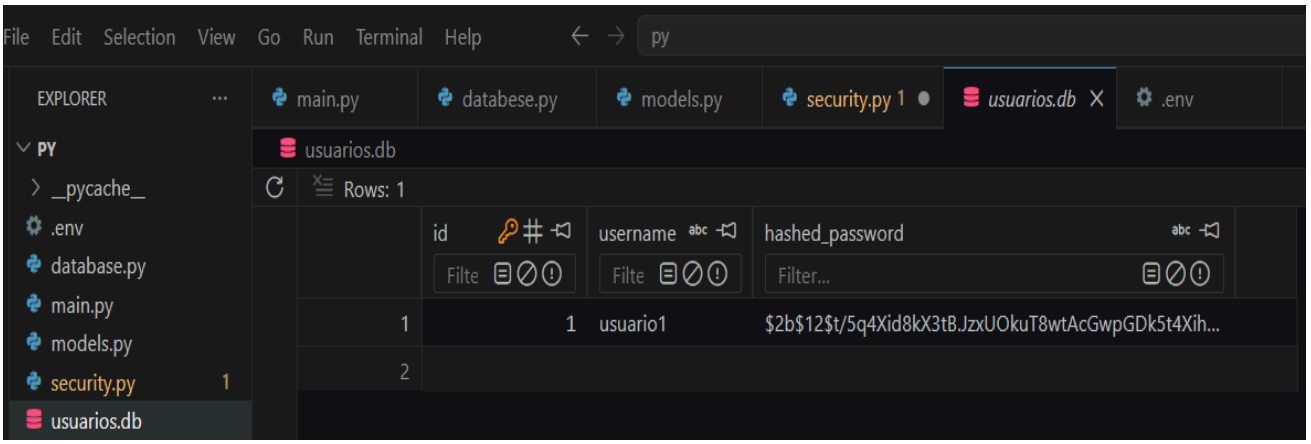
dels.py > UserAuth

```python
from sqlmodel import SQLModel, Field
from pydantic import BaseModel

class User(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    username: str = Field(unique=True, index=True)
    hashed_password: str

class UserAuth(BaseModel):
    username: str
    password: str
```

Hashing functions with Pepper integration.

```python
import os
from passlib.context import CryptContext
from dotenv import load_dotenv

load_dotenv()

PEPPER = os.getenv("PEPPER", "pimienta_por_defecto")

pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def get_password_hash(password: str) -> str:
    """Aplica el Pepper y genera el hash de la contraseña"""
    peppered_password = password + PEPPER
    return pwd_context.hash(peppered_password)

def verify_password(plain_password: str, hashed_password: str) -> bool:
    """Aplica el Pepper a la contraseña plana y la compara con el hash guardado"""
    peppered_password = plain_password + PEPPER
    return pwd_context.verify(peppered_password, hashed_password)
```

Records in the database.



3. Analysis and Reflection

What impact does Pepper have on security if the attacker manages to download the .db file but does not have access to the server's environment variables?

The impact is overwhelmingly in our favor. If a hacker steals the database, they'll only see strange text,the hashes. Normally, they'd try to use lists of common passwords dictionaries to see if the hashes match. But since we add Pepper to our passwords, the resulting hashes are completely different from those of common passwords. Because the attacker doesn't have the .env file to know which secret word we use as Pepper, it's almost impossible for them to guess the original passwords. Their attempt to steal the database is pointless.

What is the "cost" or "iterations" in the chosen algorithm and how does it help prevent brute-force attacks?

The "cost" is the number of times the algorithm repeats its mathematical operation internally to generate the hash. The higher the cost, the more demanding and slower the task becomes for the processor. This is the best defense against brute-force attacks. For a typical user, a half-second delay in verifying their login when opening an app isn't a problem. But for a hacker trying to guess a million passwords per second, that extra half-second per attempt makes their attack incredibly slow and virtually impossible to complete.